Conceptual Architecture: WezTerm

CISC 322/326 - Software Architecture

October 9th, 2026

Mohammad Moshiri Group Leader (23xv59@queensu.ca)
Nathan Beard Presenter (23xgy@queensu.ca)
Julia Lukaszun Presenter (23dxk4@queensu.ca)
Tim Rostorhuiev (23wl42@queensu.ca)
Amro Shanaah (24rfdb@queensu.ca)
Mark Talinio (23hvvd@queensu.ca)
Matthew Wong (21myw3@queensu.ca)

Abstract

This report evaluates the conceptual architecture of WezTerm, an open-source terminal emulator and multiplexer written in Rust that runs on Linux, macOS, Windows, and BSD. Using the WezTerm documentation and changelog, the system is divided into seven major components: the User Interface, Terminal Emulator, Multiplexer, Process and Domain Management, Rendering, Configuration, and Platform Abstraction. For each of these components, this report describes what it is responsible for, how the components interact with one another, and how data flows through the system.

This report found that WezTerm mainly follows a layered architecture, where Platform Abstraction handles the details of each operating system so the rest of the system does not have to. The Multiplexer can also run as a server that the user interface connects to, which gives WezTerm a client-server style. Concurrency is present whenever more than one pane is open, and Rendering, which uses the GPU, is the most performance-critical component. Looking at WezTerm's evolution, most changes stayed within a single component, allowing developers with different specialties to work in parallel. The main limitation is the development process, since most changes go through a small team of maintainers.

Introduction and System Overview

WezTerm is an open-source, cross-platform terminal emulator and multiplexer implemented in the Rust programming language. It is compatible with multiple operating systems including Linux, macOS, Windows, FreeBSD and NetBSD. In addition to providing a traditional terminal, WezTerm allows users to organize terminal sessions into panes, tabs, windows and workspaces. It can also connect the user interface to different execution environments through its multiplexer architecture.

The purpose of this report is to analyze the conceptual architecture of WezTerm and identify the major parts that make up its system. Instead of focusing on individual classes or implementation details, this analysis looks at the responsibilities of the major components and how they contribute to the overall functionality of the software. At the conceptual level, WezTerm can be viewed as a multitude of cooperating subsystems rather than one monolithic terminal program. The major parts can be divided into several major components: the User Interface, Terminal Emulator, Multiplexer, Process and Domain Management, Rendering, Configuration and Platform Abstraction. These components work together to provide the user with a functional terminal environment.

The rest of this report is sorted as follows. The Architecture section describes each component, how it has evolved, the control and data flow, concurrency, and the architectural style of WezTerm. Next, the report will look at the dependency and sequence diagrams, its use cases, external interfaces, a data dictionary and naming conventions. Finally, the conclusion and lessons learnt.

Architecture

System overview & parts

User Interface

The User Interface handles the user's interaction with WezTerm. It receives inputs from the user and provides the visual terminal windows, tabs and panes that the user interacts with.

Terminal Emulator

The Terminal Emulator manages the state of an individual terminal session. It interprets the output produced by programs on the command line and maintains the information needed to represent the terminal on the user's screen.

Multiplexer

The Multiplexer manages multiple terminal sessions and organizes them into panes, tabs, windows and workspaces. It allows users to work with several terminal sessions within the same WezTerm environment.

Process and Domain Management

The Process and Domain Management component connects terminal sessions to the program they execute. It manages the execution environment associated with a terminal and allows WezTerm to work with different domains and execution environments.

Rendering

The Rendering component is responsible for displaying the current terminal state to the user. It converts terminal information into visual output shown in the WezTerm window.

Configuration

The Configuration component allows users to customize WezTerm's appearance and behavior to their liking. This includes settings such as fonts, color schemes, key bindings and other terminal preferences.

Platform Abstraction

The Platform Abstraction component separates platform specific functionality from the rest of the system. This allows WezTerm to provide similar functionality across different operating systems without requiring the rest of the architecture to be platform specific.

System Evolution

Origins

WezTerm originally began as a personal project by Wez Furlong in 2018, but has since grown to a cross platform tool with a large community of contributors. Looking at how the program has changed over time shows how the architecture is designed to absorb change.

Release Model

The way which the developer has decided to name his releases is to first, put the Year, Month and Date right next to each other, followed by the time and the git commit hash (e.g 20240203-110809-5046fc22). Alongside these tagged releases, a continuous / nightly build is released, which is produced continually from the main branch. Rather than semantic version numbers like 1.21.1, the project has evolved through small incremental changes. It's been two and a half years since the last stable release so the nightly build is the most up to date version.

Component Evolution

When looking at the evolution of one component, ideally a change to one feature should be confined to one component. WezTerm's changelog suggests that this is largely the case.

Rendering

What rendering does is that it takes the terminal's screen state and maps them onto your monitor, and most modern terminals use the GPU to do this. To talk to the GPU, you need to talk to it via an API. OpenGL is the old way to do this, which is widely supported. In January 2024, the default frontend was swapped from OpenGL to WebGpu, a newer API that works on top of modern graphics systems such as Vulkan, Metal, and DirectX 12. In the very next release, OpenGL became the default

option and there is an option to change and users can choose the WebGpu through the “front_end” settings. This is possible due to the Terminal Emulator only tracks what should be on the screen, while rendering decides how to actually draw it. The graphics backend can be changed without affecting the other components.

Process & Domain Management

The domain determines where a pane’s program gets executed, such as a local machine, or remoting over SSH, or a WezTerm multiplexer server. Recently, WezTerm added support for tmux control mode, which allows tmux sessions to appear on the WezTerm panes. Since this did not require any changes to how the panes / tabs actually worked, and since the Multiplexer doesn't need a domain to connect, a new way of running a session was added without touching the core of the system.

We can see similar patterns appear in other places too, such as the wayland support being rewritten within the Platform Abstraction layer without altering the rest of the system, or the Lua config support that continues to gain features like plugins.

System control and data flow

Overall Control Model

WezTerm control model has multiple areas to it, the main ones being user input, running terminal processes, config/Lua events, and operating-system events. The UI captures user interaction, while the multiplexer determines where the operation should be applied to, such as different panes, tabs or windows to workspaces or terminal sessions. The execution of programs and connections to local and remote environments is handled through process and domain responsibilities. Terminal output is interpreted into terminal state before being passed to the rendering component for presentation.

User Input and Control Flow

Keyboard and mouse events enter WezTerm through the graphical/windowing environment. Configuration determines whether an input is supposed to correspond to a WezTerm action or if it should be forwarded to the running terminal program. As an example, a key binding could request an operation such as splitting a pane or switching tabs, in which case, control is directed towards the multiplexer. If the input is instead intended for the running application, it is first encoded as a terminal input and then routed to the currently active pane and its associated execution environment.

Process Output and Data Flow

The data flow is in the opposite direction when a running program produces an output. The application writes a byte stream through its pseudo-terminal (PTY), which can contain both printable characters and terminal escape or control sequences. The terminal emulator interprets these sequences and updates the terminal state that is associated with the pane. The rendering component then uses this state to generate the graphical representation displayed through the UI.

Running process -> Process and Domain management / PTY -> Terminal Emulator -> terminal state -> Rendering -> UI -> User

Multiplexer and Session Control

The multiplexer controls operations that change the organization of the terminal sessions, including panes, tabs, mux windows and workspaces. User actions (such as splitting a pane or creating a tab) flow from the UI to the multiplexer. Some of these actions also trigger process and domain operations; for example, splitting a pane normally creates a new pane and launches a command in it, while switching between existing panes mainly changes the multiplexer state.

Configuration and Lua Flow

The configuration component provides policy and config data to other subsystems through “wezterm.lua”. When it’s evaluated the configuration affects input mappings, fonts, colours, rendering options, the default program, and the domain used for new sessions. Lua callbacks work slightly differently, since they can actively participate in runtime control. Registered event handlers are able to react to events and perform actions, including launching commands and creating windows.

Local and Remote Domain Flow

Domains allow the same architecture to support different execution environments. In the local domain (default), the multiplexer connects a pane to a locally spawned process. In an SSH multiplexing domain, the local GUI instead connects to a remote WezTerm multiplexer, which allows remote panes, tabs and windows to appear in the local interface. This means that the presentation side can remain in a local environment while the mux session and executed process may exist remotely.

System concurrency

Concurrency is present in WezTerm whenever multiple panes, tabs or windows are open, because each pane runs its own program at the same time. The main units of concurrency are the running programs, the multiplexer server, and any connections

to remote domains. WezTerm handles each session's output independently so a busy pane does not freeze any of the UI or other panes, and the multiplexer server can keep sessions running with no window open.

Performance Critical Components

For WezTerm, rendering is the most performance critical component since it must be able to redraw anything happening on the window and to make everything feel as smooth as possible. This is why WezTerm uses the GPU. In addition to this, the Terminal Emulator must keep up with large amounts of output and the remote domains depend on the speed of the network.

Architectural Style

WezTerm mainly follows a layered architecture style. It separates responsibilities between the User Interface, terminal and session management, rendering and platform specific services. This reduces the coupling between responsibilities and it also makes each individual part easier to change. This is great when a large change needs to be done to a part, like rendering, but you don’t want to redesign the entire system. However, the layers are not fully independent since components still need to interact with each other, like the Multiplexer and Configuration often do.

WezTerm follows a client-server style since its built in multiplexer can run as a server that the UI connects to, locally or remotely, without needing tmux[4]. This lets the sessions keep running after the window closes, but also adds complexity and makes performance depend on the network when remote.

A more monolithic design could simplify communication between all these responsibilities, but has the trade-off that increases coupling between many of the components such as, the graphical interface, terminal sessions and execution environments. This makes features like remote multiplexing more difficult to develop, and makes it significantly harder to update individual components.

Testability

WezTerms' architecture style directly helps its testability. Since the responsibilities are separated, major areas can be tested and validated with fewer unrelated dependencies. This makes faults and errors easier to isolate than in a monolithic design. However, interactions between several components are still present and cannot be isolated, so something like, a user input going to the Multiplexer to a process still requires integration testing to ensure that components are able to communicate directly.

Implications and division of responsibilities

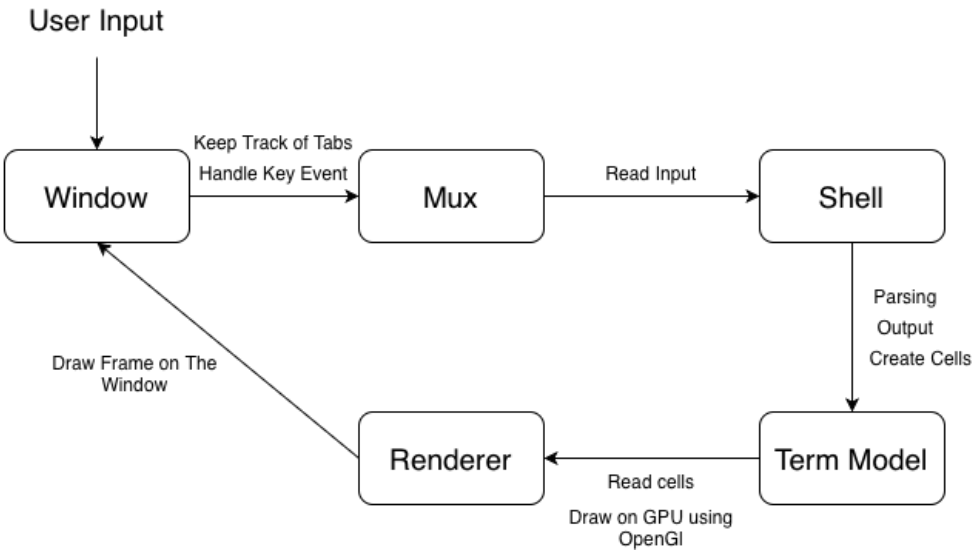
Each component of WezTerm requires a different area of expertise. For example, Platform Abstraction needs specific knowledge of how each operating system handles inputs and windows, while domains require networking knowledge for connecting to other machines using SSH. This allows all the work to be divided based on the specialty of each developer, each of them working on a different component at the same time and coordinating where each part connects together. The main limiting factor is that most changes are reviewed by a small core of maintainers, so this approval process can become a bottleneck as more developers contribute.

Diagrams

Derivation Process

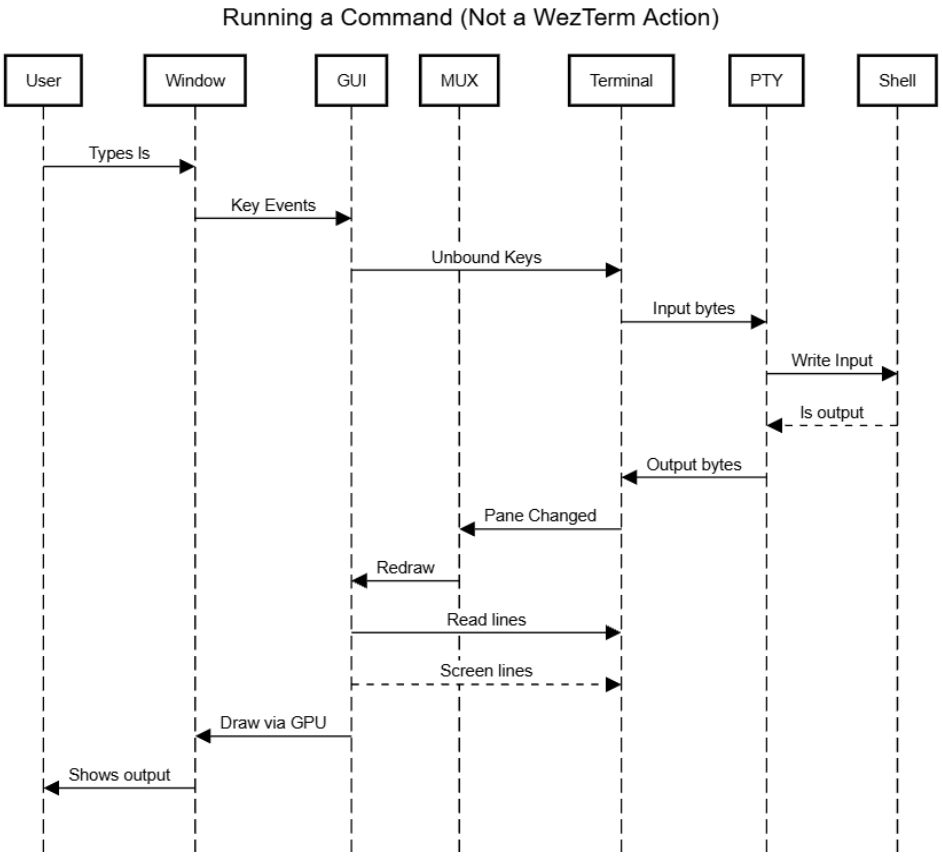
The conceptual architecture can be derived in three steps; first, the team read WezTerm's documentation & changelog [1][3][4], in addition to the general overview of how the terminals actually worked [7] to find the system's main responsibilities. Next, those responsibilities were used to find common use cases such as running a command and opening new tabs, through the system, and were made into sequence diagrams. Finally, the components involved in these sequence diagrams were made into a dependency diagram to show each component depending on the others.

Dependency Diagram



This diagram traces one keystroke in a loop, starting from the user input where once you type, the window receives it. Upon receiving key input, it handles it to the mux, which reads the input and routes it to the shell. The shell parses the text then hands it to the Term Model, which creates a grid of cells, where each cell holds a character and attribute. After that, the term model hands the cells to the Renderer which reads each cell, draws the grid on the GPU and sends the frame back to the Window.

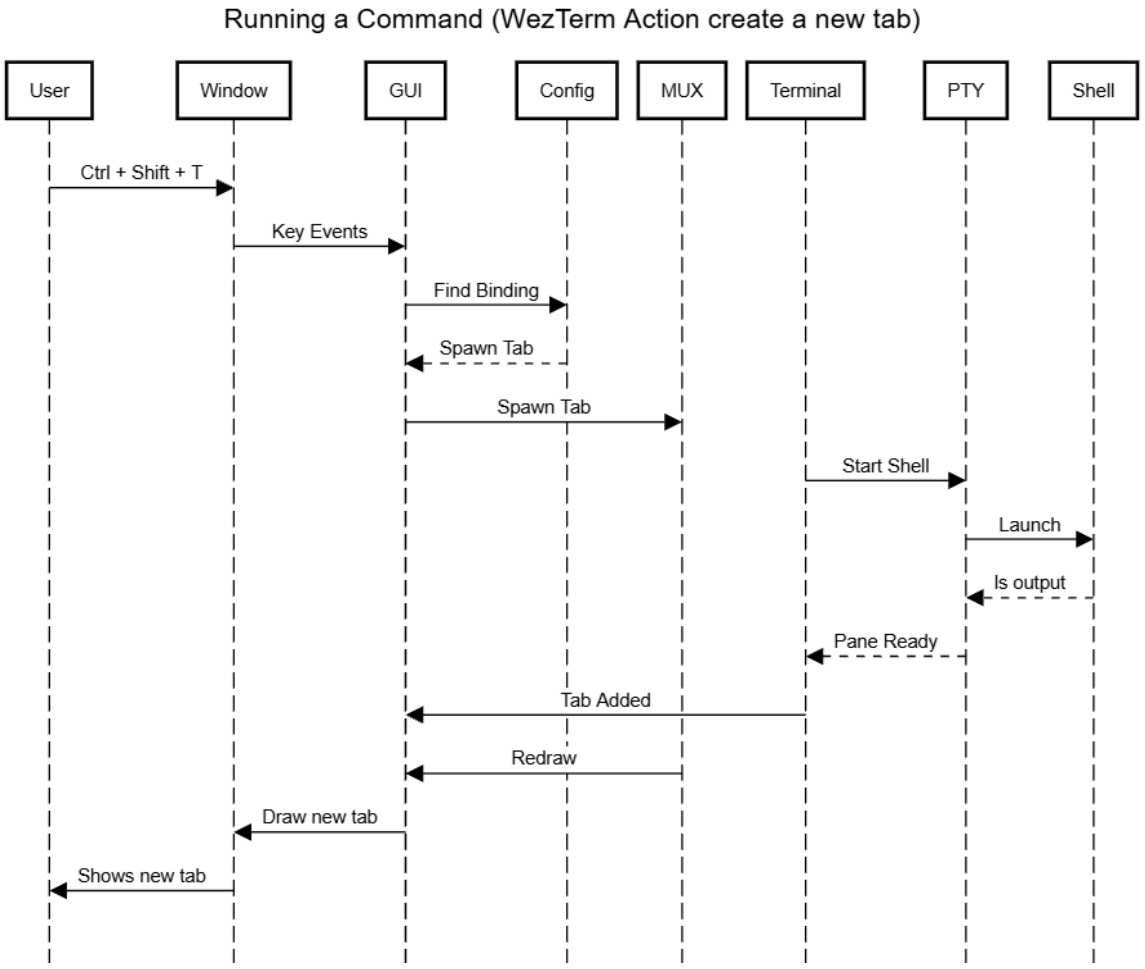
Legend: boxes at the top are components or actors; solid arrows are messages or calls; dashed arrows are returned responses.



The sequence diagram goes over when the user runs the list command. The user inputs their keystrokes which passes through the window subsystem to the GUI. The keys that are not bound to a WezTerm action go to the Terminal subsystem, which converts them into bytes. The PTY subsystem takes those input bytes and writes them to the Shell.

The shell then returns output through the PTY to the Terminal subsystem, updating the screen model. The Terminal communicates with the MUX about the changes to the pane, which causes the MUX to tell the GUI to redraw. The GUI reads the changed lines and draws them through the GPU.

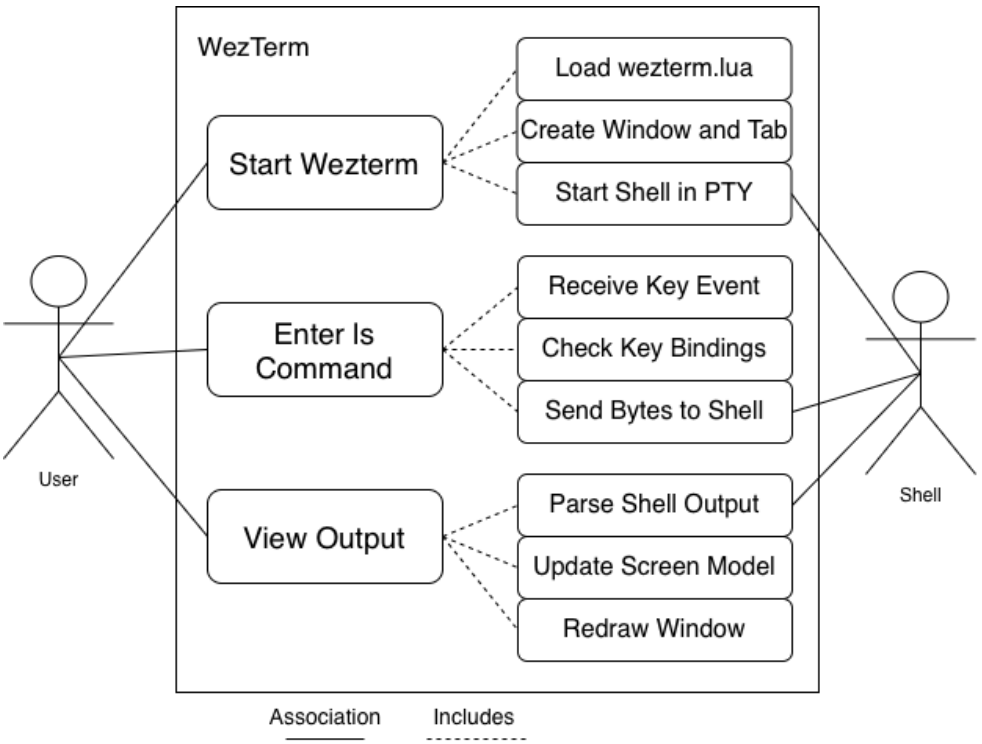
Legend: boxes at the top are components or actors; solid arrows are messages or calls; dashed arrows are returned responses.



This sequence diagram goes over what occurs when the user inputs a key event to spawn a new tab. The Window subsystem passes the input to the GUI as a key event. The GUI searches for the key in the Config, finding this key event bound to the SpawnTab action. The GUI carries out this action by communicating to the MUX to create a new tab. The MUX has the PTY subsystem start a new shell in the tab's pane. The MUX tells the gui that a tab was added, and the GUI draws the updated tab bar.

Use Case

The diagram below displays the use case:



The use case that would be most common on WezTerm is a user running a command. We will use the ls command for our use case. The user opens WezTerm and enters the “ls” aiming to list the directories and files within their current directory. When WezTerm starts, the configuration is read from the wezterm.lua in order to load the users fonts, color and keybindings. The multiplexer then creates a window with a single tab and the PTY subsystem starts the user’s shell inside it.

The user inputs from the keyboard and the window subsystem receives it from the operating system turning it into a key event. The GUI checks if the key event is bound to a WezTerm action. If it is not, the key is passed through the terminal emulation subsystem, which converts it to bytes. Those bytes are sent to the shell by the PTY subsystem.

The shell then writes the output of the command, which is the list of directories and files back to the PTY subsystem. This output goes to the terminal emulation subsystem it reads the text and any color codes attached to it, updating the screen model with the new characters, colors and cursor position. It then communicates with the multiplexer about the changes to the pane.

The GUI is notified about the changes and redraws the window. It reads the updated lines from the screen model, asks for the font subsystem for the right glyphs, and draws them through the GPU. The user sees the produced list that is followed by a new prompt.

External Interface

WezTerm puts information through a set of different systems in order to process user inputs, manage internal terminal processes, configuration files, and remote machines in a network.

User Input Handling

Input from the user comes in through with physical tools such as a keyboard and mouse. These inputs are received by the window subsystem first, which interfaces over Windows, MacOS, X11 and Wayland turning OS events into a set of key and mouse events.

Those events are then passed into the GUI subsystem, which checks the inputs against the user's key bindings. The matching event triggers a WezTerm action like opening a tab, splitting panes or copying text. Other events are passed to the terminal emulation subsystem, which encodes it as a byte sequence a terminal program expects. The PTY subsystem then writes those bytes to the running programming.

Outputs from the program travel the other way. The PTY subsystem reads the program's output as a stream of bytes and passes through to the terminal emulation subsystem. That subsystem interprets the text and escape sequences to update its model of the screen, which are the characters, colours, cursor position, hyperlinks and images.

Output to the user is handled by the GUI subsystem. Taking the screen model, uses the font subsystem to look up and shape glyphs from the system's font files, and returns the result to the user's monitor through their GPU, using OpenGL or WebGPU. This same subsystem shows the tab bar, status line, search overlay and selection. It can also pass copied text to the OS clipboard as well as pass notifications to the desktop.

Configuration

Configuration is read from a Lua File, wezterm.lua, by the configuration subsystem when WezTerm starts. This file is reloaded automatically if ever changed. It provides the settings to the interface's fonts, colors, key bindings and remote host definitions. Through the scripting interface, the configuration also responds to events from WezTerm and requests actions in return.

Remote Systems

Remote sessions are functional because of the multiplexor. The user connects to a remote host, the SSH subsystem authenticates the user and checks the host identity. The multiplexer client proceeds to send messages with the multiplexer server on the remote machine. This server sends the layout of the windows, panes, and any changes made to them.

Local connections use a Unix socket, while remote connections use TLS or SSH [4]. These remote sessions continue to persist even after the local system has dropped connection.

Data Dictionary & Naming Conventions

AI - Artificial Intelligence, a tool used to perform tasks that usually require human intelligence.
API - Application Programming Interface, a defined interface that allows software components or programs to communicate with each other and use each other's functionality
Backend - The parts of the software system that are not directly visible to the user
Client-Server Architecture - An architectural style in which a client communicates with a server to request services or information
Command Line - A text based interface that allows users to interact with the computer with text input
Configuration - Settings that determine how a system behaves and appears, including fonts, colours and key bindings
MUX - Multiplexer, the component that manages terminal sessions and organizes them into panes, tabs, windows and workspaces
Frontend - The parts of the software system that the users directly interact with
GPU - Graphics Processing Unit, hardware used to accelerate graphics operations
GUI - Graphical User Interface, visual interface through which users interact with software
Layered Architecture - An architectural style that separates a system into layers with distinct responsibilities
Lua - A scripting language used by WezTerm for configuration and runtime event handling
Pane - a section of a WezTerm window or tab that displays a separate terminal session
PTY - Pseudo-Terminal, a software interface that allows a terminal application to communicate with a program as if it were connected to a physical terminal

SSH - Secure Shell, a protocol used to securely connect to and communicate with remote systems
Terminal - An interface that allows users to interact with a command-line environment and execute commands
TLS - Transport Layer Security, a protocol that provides encryption and authentication for network communications
tmux - A terminal multiplexer that allows multiple terminal sessions to run within a single terminal window
UI - User Interface, the part of the system through which a user provides input and receives output

Release Names
YYYYMMDD-HHMMSS-commithash

- Date (year, month, day): 20240203
- Time (hours, minutes, seconds): 110809
- Git commit hash: 5046fc22

Conclusions

In conclusion, WezTerm's architecture is layered, with Platform Abstraction separating each operating system's details from the rest of the system, while the multiplexer domains give WezTerm a client-server style which allows sessions to run locally, remotely, or in the background. In its evolution, most changes happened within a single component, which makes it easier for developers with different specialties to work in parallel with one another. The main limitation of WezTerm is the development process itself since most changes go through a small team of maintainers. In the next part of this assignment, WezTerm's source code will be compared against this conceptual architecture to see if the concrete architecture matches the documentation.

Lessons Learned

One of the biggest lessons that we learned from doing all of this research and analysis of WezTerm is that making a conceptual architecture from documentation is not a simple task. There are so many things that we needed to look at, for example, a pane is managed by the Multiplexer but it’s also connected to a running process and it also contains a terminal state. Something else we learned is how important it is to separate concepts that may seem like they are similar and could be grouped together. These were things like terminal emulation and rendering or a mux window and a graphical window. We think that if we were to repeat this process on a new

piece of software, we would first figure out the major architectural components, concepts and relationships earlier in the process so that when we dive into more detailed sections, it would be a lot easier to keep component boundaries and terminology more consistent throughout the report.

References

[1] https://wezterm.org/index.html
[2] https://wezterm.org/config/files.html
[3] https://wezterm.org/changelog.html
[4] https://wezterm.org/multiplexing.html
[5] https://github.com/wezterm/wezterm/releases
[6] https://linuxcommandlibrary.com/man/wezterm
[7] https://wezterm.org/what-is-a-terminal.html
[8] https://wezterm.org/config/lua/wezterm.mux/index.html
[9] https://wezterm.org/config/lua/pane/index.html
[10] https://wezterm.org/config/lua/mux-window/index.html
[11] https://wezterm.org/config/lua/keyassignment/AttachDomain.html
[12] https://wezterm.org/config/lua/config/front_end.html

AI Collab Report

AI Member Profile and Selection Process

Our AI teammate was Anthropic's Claude (Opus 5.5), used from the 25th of September to the 9th of October 2026. WezTerm's documentation is spread across hundreds of separate pages (config options, Lua APIs, a changelog covering years of releases), and no single page describes or summarizes the overall architecture, so we needed a tool that could read long pages such as Multiplexing [4] and the changelog, explain new terms (PTY, domain), and quote the source for each answer. We tested ChatGPT on the same question about domains, Claude quoted the docs more reliably, so we chose it. AI was treated as a supporting resource and not a main source, because conclusions about a software system needed to be consistent with its documentation and changelog and AI can always hallucinate incorrect information to base a conclusion off.

Tasks Assigned to the AI Teammate

We limited AI to a small supporting role in research and analyzing:

- Architectural analysis: organizing the system into responsibilities (User Interface, Terminal Emulator, Multiplexer, etc.), since AI can summarize

complex technical information into straightforward and easily understandable categories.

- Control and data flow reasoning: thinking through how user input reaches a running process, how output updates terminal state, and how that state is rendered.
- Clarification of concepts: explaining multiplexing, local and remote domains, terminal emulation and rendering, so these distinctions and differences get documented in our report.
- Drafting and revision: first drafts of several sections, which team members edited and fact-checked.

AI was not a substitute for the team's research or diagrams, and all AI drafted text was edited and verified by team members before inclusion.

Interaction Protocol and Prompting Strategy

Amro coordinated the group's AI use as the prompt engineer, members brought their questions to him, he turned them into detailed prompts and shared the answers. We used AI for targeted questions rather than whole sections, and treated every answer as material to be checked and verified.

Our first prompt was general: "Explain the architecture of WezTerm." It gave a generic layered terminal that could describe almost any terminal emulator. We improved it by adding a persona, our component list, the documentation as context, a required structure, and a prompt to flag claims Claude was unsure of:

> "*Act as a software architecture engineer studying WezTerm. Using the attached documentation and sources, explain the responsibilities of the User Interface, Terminal Emulator, Multiplexer, Process and Domain Management, Rendering, Configuration, and Platform Abstraction. Then trace a normal command from keyboard input to the running process and back to the rendered screen. Distinguish control flow from data flow, explain how local and remote domains are different, and identify which claims require confirmation from the official documentation or changelog.*"

Validation and Quality Control Procedures

Because AI explanations can be believable without being accurate, verification was crucial. We used the project's primary documentation and changelog as references for checking architectural claims. Our report's reference list includes the official WezTerm documentation, configuration documentation, multiplexing documentation, changelog, and release information.

The quality control process was to:

Check technical claims with source material. Claims about configuration, multiplexing, terminal behavior, and remote connections should be compared with the official documentation rather than accepted just because an AI explanation sounds right. Retain human responsibility for conclusions. The team remains responsible for deciding which information is relevant and ensuring that the conclusion represents a human interpretation of WezTerm, not one made by AI that might give inaccurate information.

We described remote connections incorrectly as using one type of socket after Claude told us. After checking WezTerm's documentation, we corrected this to distinguish Unix sockets, TLS, and SSH as three different connection methods

Quantitative Contribution to the Final Deliverable

We estimate the AI's overall collaboration at about 9%. Its collaboration was limited to supporting technical understanding and definitions, organizing ideas, and explaining component relationships. The remaining work involved researching and analyzing source material, determining the scope of the conceptual architecture, developing the report and diagrams, coordinating the presentation, and taking responsibility for the final conclusions.

Reflection on Human–AI Team Dynamics

AI reduced the time needed to understand new concepts, but it did not remove the need for research. An explanation from AI can hide important distinctions or describe a design the system does not have, so checking against the documentation was necessary and used up part of the time saved.
AI worked best as an easy explanation and brainstorming resource, not as the decision maker. The team still decided which components mattered, how their responsibilities get separated, and how to display the evidence in a coherent and easy to understand manner.